

Salesforce Developer Bridge Program

Day 1 Task Submission Report

Submitted By: Midasala Nikshitha Ramesh [23PA1A05F2]

Project: Hospital OPD Management System

Task 1 – Design a Mini Data Model

Objective:

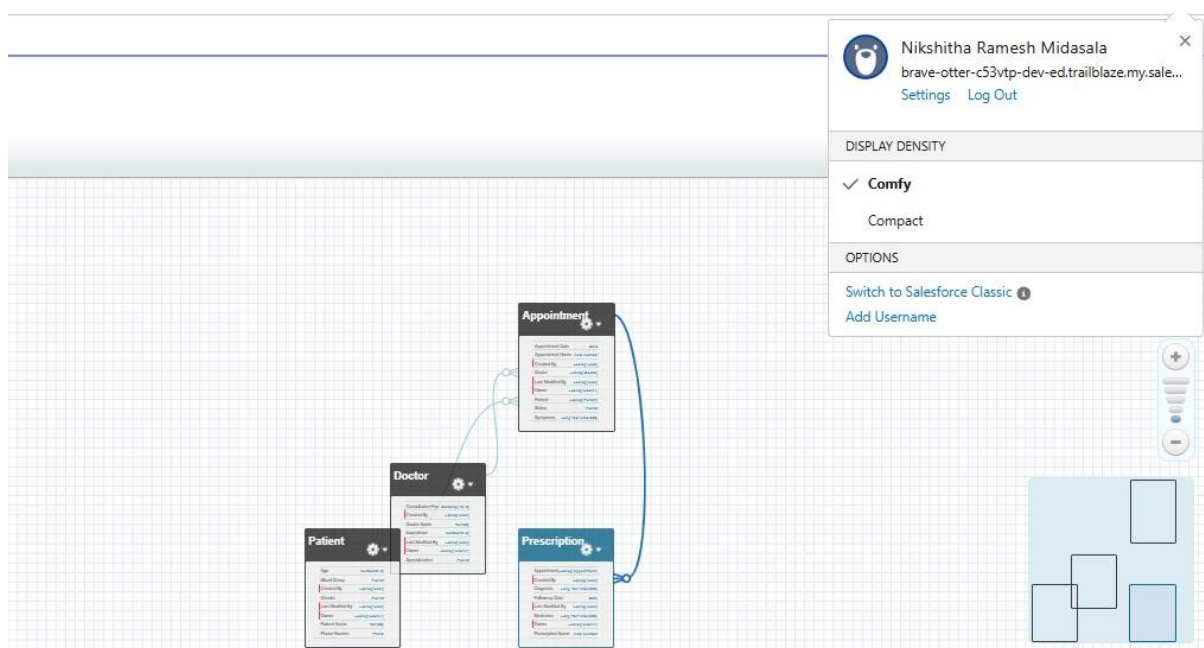
To design and implement a custom Hospital OPD data model in Salesforce by creating custom objects, fields, and relationships to represent real-world healthcare data.

Work Performed:

- Created custom objects namely **Patient**, **Doctor**, **Appointment**, and **Prescription**.
- Added the required custom fields for storing patient, doctor, and appointment information.
- Established relationships between the objects using Salesforce relationship fields.
- Verified the complete data model using **Schema Builder**.

Outcome:

Successfully created the Hospital OPD data model with all required custom objects and relationships. The data model was validated in Salesforce and was ready for further development.



Task 2 – Apex Basics

Objective:

To understand the fundamentals of Apex programming by writing and executing Apex code using the Salesforce Developer Console.

Work Performed:

- Opened the Developer Console and used the **Execute Anonymous** window.
- Executed sample Apex code to understand the execution process.
- Verified the execution using the generated debug logs.
- Reviewed the execution details to understand Apex program flow.

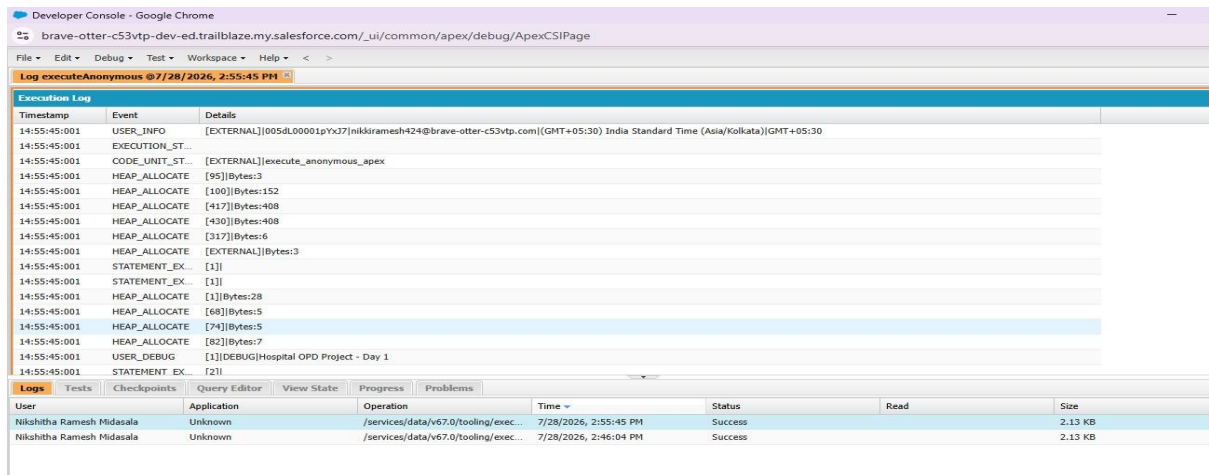
Outcome:

The Apex code executed successfully without errors, and the debug log confirmed the successful execution of the program.

Execute Anonymous Window:



Debug Log:



Task 3 – SOQL Practice

Objective:

To gain practical knowledge of Salesforce Object Query Language (SOQL) by retrieving records from custom objects using different query techniques.

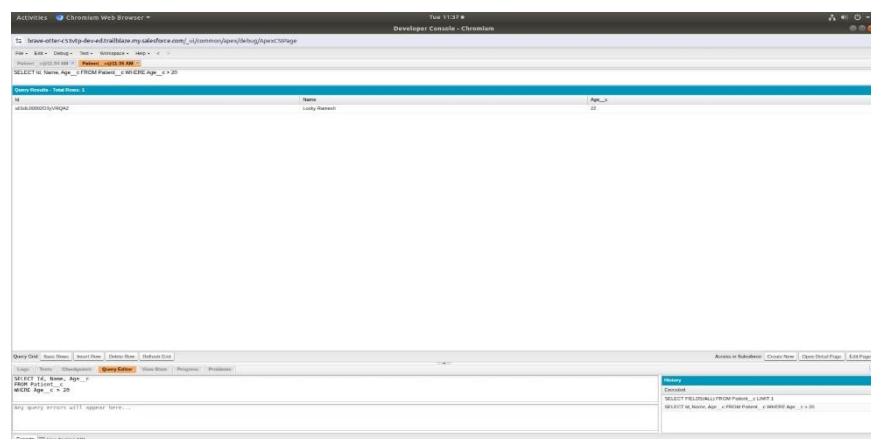
Work Performed:

- Executed **WHERE** clause queries to filter records.
- Used **ORDER BY** and **LIMIT** to sort and restrict query results.
- Performed relationship queries to retrieve related object information.
- Executed aggregate queries using **COUNT()** and **GROUP BY**.
- Used comparison operators to retrieve records based on specific conditions.

Outcome:

Successfully executed all required SOQL queries and obtained the expected results without any syntax errors, improving understanding of Salesforce data retrieval.

WHERE Clause Query:



ORDER BY + LIMIT Query:

The screenshot shows the Chrome DevTools Console with a query executed. The query is:

```
SELECT Hl.Name, Consultation_Fee_c FROM Doctor_c ORDER BY Consultation_Fee_c DESC LIMIT 1
```

The results table shows one row:

Hl	Name	Consultation_Fee_c
403630000464Q2Q2	Dr. Noemima Rindiy	800

The query editor shows the following code:

```
SELECT Hl.Name, Consultation_Fee_c FROM Doctor_c ORDER BY Consultation_Fee_c DESC LIMIT 1
```

Any query errors will appear here...

Relationship Query:

The screenshot shows the Chrome DevTools Console with a query executed. The query is:

```
SELECT Name, Appointment_Date_c, Status_c, Patient_c, Name, Doctor_c FROM Appointment_c
```

The results table shows two rows:

Name	Appointment_Date_c	Status_c	Patient_c	Name	Doctor_c	Name
AP1-0001	2020-01-01	Scheduled	None	None	Dr. Noemima Rindiy	
AP1-0002	2020-01-02	Scheduled	Lady Noemima	None	Dr. Noemima Rindiy	

The query editor shows the following code:

```
SELECT Name, Appointment_Date_c, Status_c, Patient_c, Name, Doctor_c FROM Appointment_c
```

Any query errors will appear here...

Aggregate Query:

The screenshot shows the Chrome DevTools Console with a query executed. The query is:

```
SELECT Status_c, COUNT(*) FROM Appointment_c GROUP BY Status_c
```

The results table shows one row:

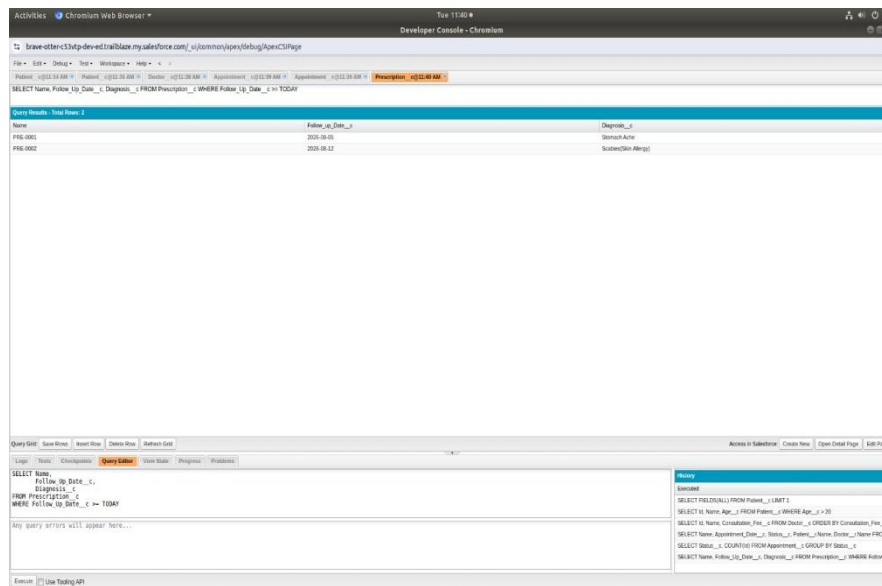
Status_c	count(*)
Scheduled	2

The query editor shows the following code:

```
SELECT Status_c, COUNT(*) FROM Appointment_c GROUP BY Status_c
```

Any query errors will appear here...

Comparison Operator Query:



The screenshot shows a Salesforce Developer Console window. At the top, the browser address bar displays the URL: `https://brave-051314p-dev-ed3ralilake.my.salesforce.com/_ui/common/apex/debug/ApexCSPage`. Below the browser window, the console shows a SOQL query: `SELECT Name, Follow_Up_Date__c, Diagnosis__c FROM Prescription__c WHERE Follow_Up_Date__c <= TODAY`. The query results are displayed in a table with three columns: Name, Follow_Up_Date__c, and Diagnosis__c. The results show two records: one for 'PAC-0001' with a follow-up date of '2020-08-01' and diagnosis 'Stomach Ache', and another for 'PAC-0002' with a follow-up date of '2020-08-12' and diagnosis 'Stomach Pain (Abdominal)'. The bottom of the console shows the query editor with the same SOQL query and a 'Run' button.

Name	Follow_Up_Date__c	Diagnosis__c
PAC-0001	2020-08-01	Stomach Ache
PAC-0002	2020-08-12	Stomach Pain (Abdominal)

Task 4 – Apex Trigger

Objective:

To implement an Apex Trigger using a separate Handler Class for validating records and automating business processes.

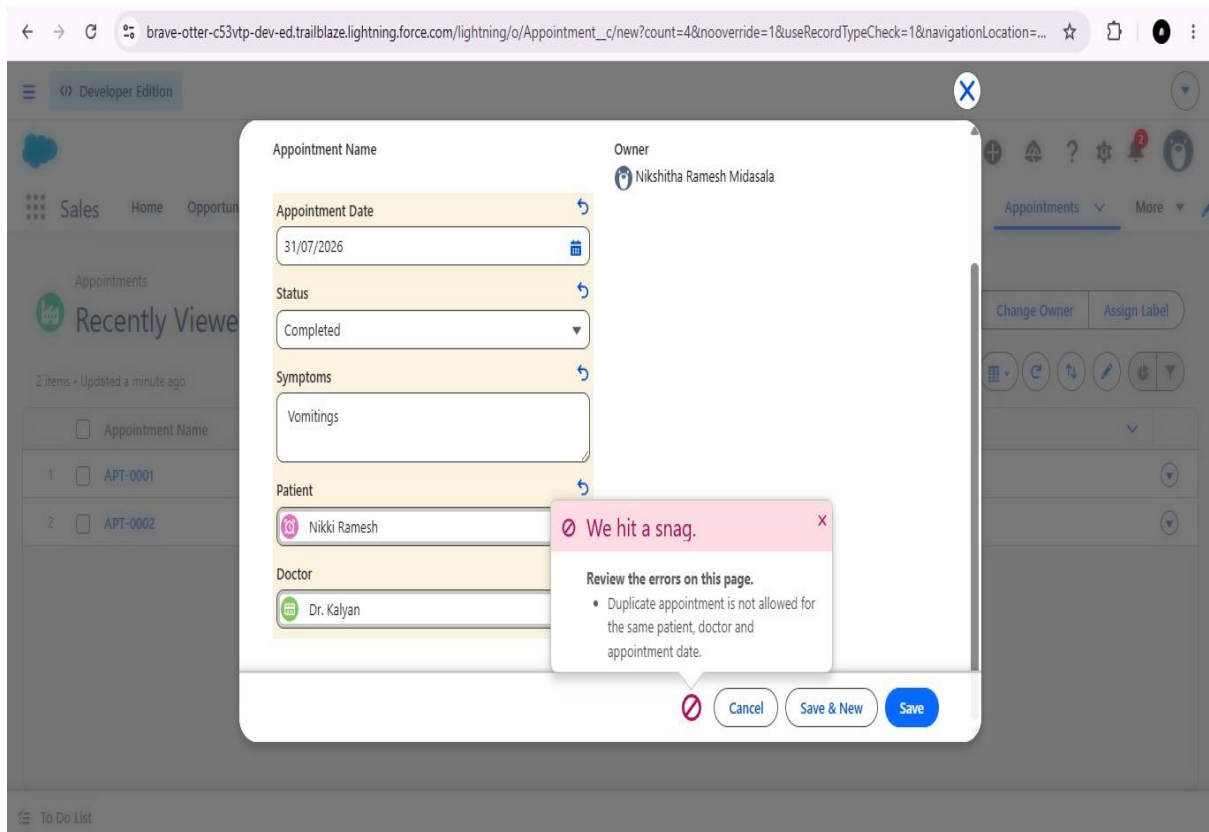
Work Performed:

- Created an Apex Trigger for the **Appointment** object.
- Implemented the trigger logic inside an **AppointmentTriggerHandler** class.
- Added validation to prevent duplicate appointments for the same patient, doctor, and appointment date.
- Implemented logic to update related prescription records after appointment completion.
- Tested the trigger by creating and updating appointment records.

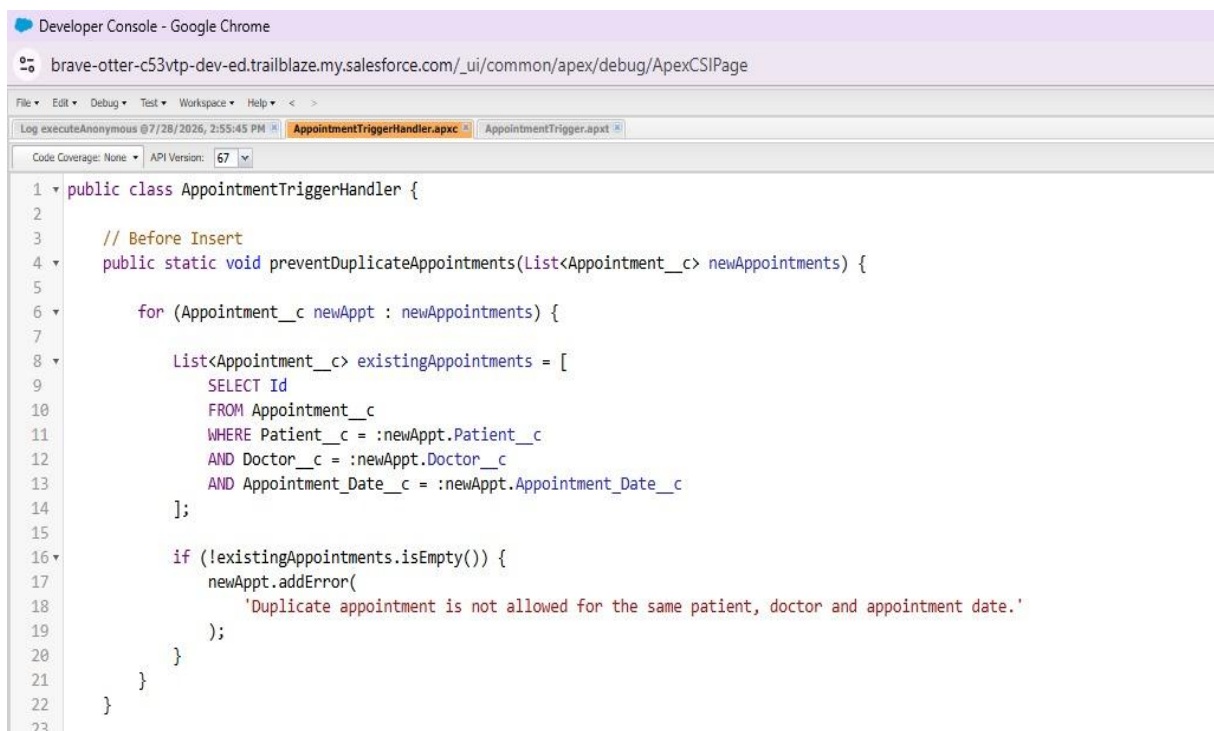
Outcome:

The trigger successfully prevented duplicate appointment entries and updated related records automatically, demonstrating proper trigger execution and Salesforce best practices.

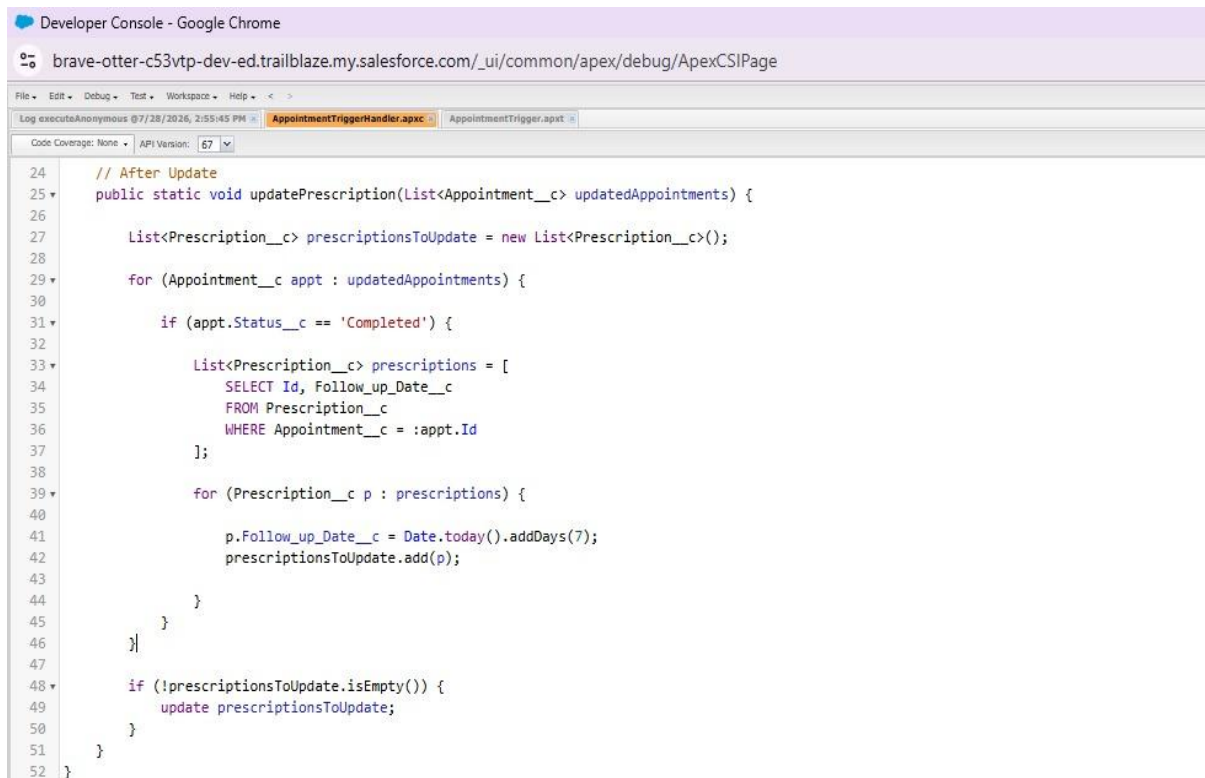
Duplicate Appointment Validation:



Before Insert Handler Code:



After Update Handler Code:

The image is a screenshot of the Salesforce Developer Console. At the top, the browser address bar shows the URL: brave-otter-c53vtp-dev-ed.trailblaze.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage. Below the address bar, the console title is "Developer Console - Google Chrome". The main area displays the Apex code for a class named "AppointmentTriggerHandler.apxc". The code is as follows:

```
24 // After Update
25 public static void updatePrescription(List<Appointment__c> updatedAppointments) {
26
27     List<Prescription__c> prescriptionsToUpdate = new List<Prescription__c>();
28
29     for (Appointment__c appt : updatedAppointments) {
30
31         if (appt.Status__c == 'Completed') {
32
33             List<Prescription__c> prescriptions = [
34                 SELECT Id, Follow_up_Date__c
35                 FROM Prescription__c
36                 WHERE Appointment__c = :appt.Id
37             ];
38
39             for (Prescription__c p : prescriptions) {
40
41                 p.Follow_up_Date__c = Date.today().addDays(7);
42                 prescriptionsToUpdate.add(p);
43
44             }
45         }
46     }
47
48     if (!prescriptionsToUpdate.isEmpty()) {
49         update prescriptionsToUpdate;
50     }
51 }
52 }
```

Task 5 – Lightning Web Component (LWC)

Objective:

To develop and deploy a Lightning Web Component that displays patient information from Salesforce using Apex and the @wire service.

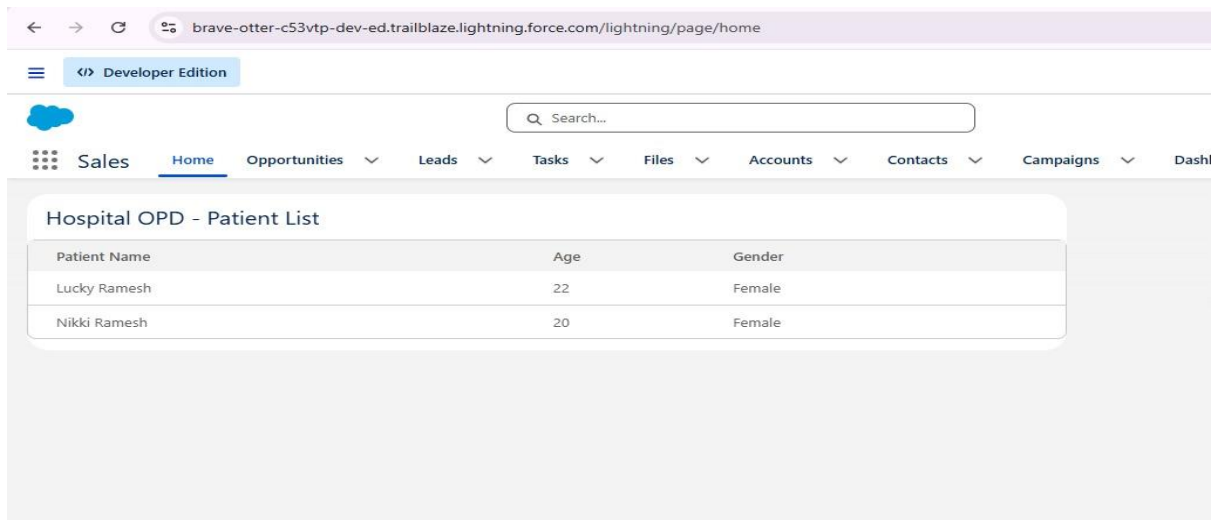
Work Performed:

- Created a custom Lightning Web Component named **patientList**.
- Developed the HTML, JavaScript, and metadata configuration files.
- Connected the component with an Apex controller to retrieve patient records.
- Deployed the component to the Salesforce Developer Org using Visual Studio Code.
- Added the component to the Salesforce Home Page and verified its functionality.

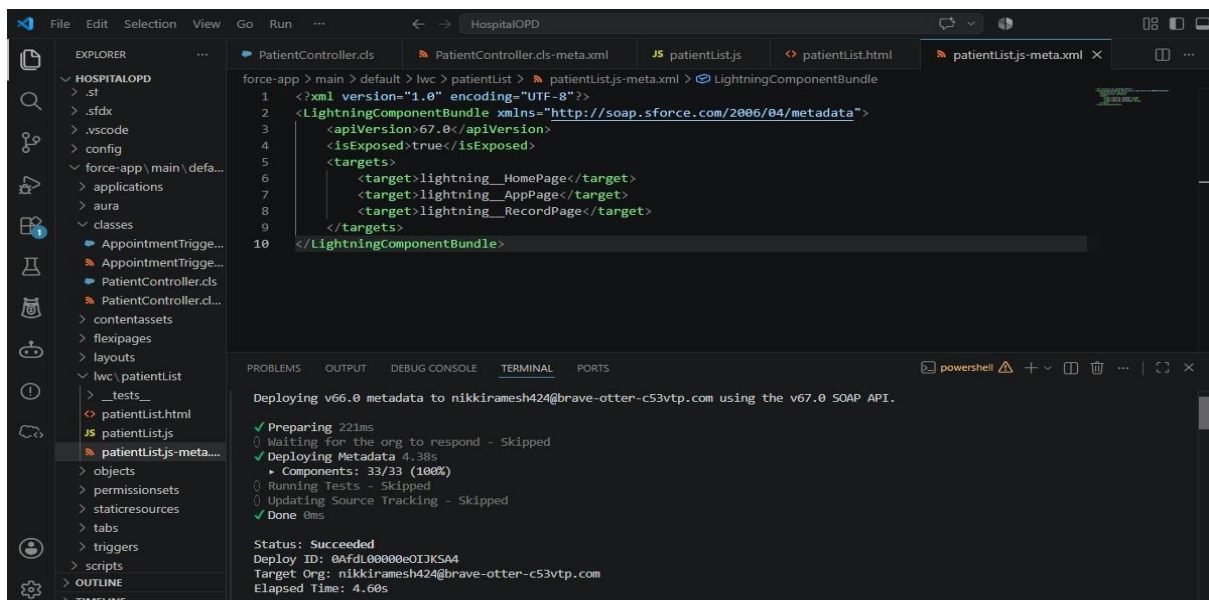
Outcome:

The Lightning Web Component was deployed successfully and displayed patient records on the Salesforce Home Page, confirming successful integration between Apex and LWC.

Patient List Component on Home Page:



Successful Deployment in VS Code:



Conclusion:

Successfully completed all five Day 1 tasks and gained practical knowledge of Salesforce development, including data modeling, Apex, SOQL, Triggers, and Lightning Web Components.